

SLIIT CODEFEST 2026 - AI COMPETITION

Intelligent Document Assistant

Team: NextGen

Chosen Sub-track: 1B - Connecting Facts Across Thousands of Pages

Demo Video: [Click Here](#)

1. Problem Statement

The Ashen Era Archive contains **415 documents and approximately 1,277 pages** across PDF, DOCX, Markdown, plain text, simulated scans, novels, wiki articles, codexes, ephemera, and images. Information about the same character, organization, event, location, or artifact may appear across several different documents. For many questions, retrieving a few semantically similar passages is not enough because **no single document contains the complete answer**. The system must discover related facts across documents and connect them before generating an answer. We therefore selected:

Sub-track 1B - Connecting Facts Across Thousands of Pages.

Our goal was to build an AI document assistant capable of retrieving and connecting distributed evidence from the archive rather than relying only on conventional single-pass semantic search.

2. Solution Overview

Our solution uses Hybrid Graph + Vector Retrieval Architecture. The system first loads and preprocesses documents from the archive. The processed content is then represented in two complementary ways:

- Vector representation for semantic similarity search.
- Graph representation for discovering entities and relationships across documents.

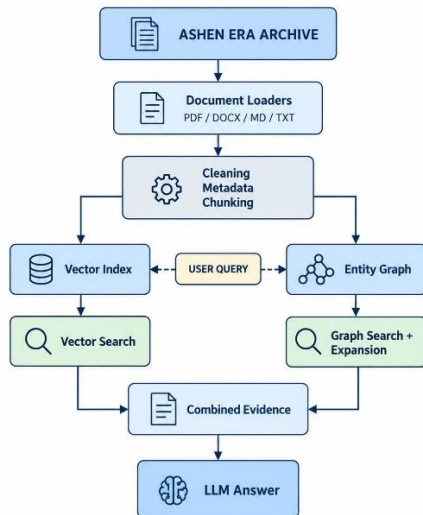
When a user submits a question, the system performs both semantic retrieval and graph-based retrieval. Graph retrieval identifies related entities and expands their connections, while vector retrieval finds relevant document chunks.

The main principle of our approach is:

Vector retrieval finds relevant text; graph retrieval connects related facts.

This directly addresses Sub-track 1B, where answers may require evidence distributed across multiple documents.

3. Architecture & Technical Decisions



4. Key Technical Decisions

4.1 Hybrid Retrieval Instead of Vector-Only RAG

Traditional RAG retrieves passages independently according to semantic similarity. This works well when the answer exists directly inside one or two chunks, but Sub-track 1B specifically requires connecting information spread across documents.

We therefore combined **semantic vector retrieval with graph relationship retrieval**.

4.2 Modular Document Ingestion

Because the archive contains multiple document formats, we implemented separate loaders:

- loader.py
- pdf_loader.py
- docx_loader.py
- markdown_loader.py
- text_loader.py

This keeps format-specific parsing separate and makes the ingestion pipeline easier to maintain and extend.

4.3 Separate Preprocessing Components

Cleaning, chunking, and metadata generation were implemented separately:

- cleaner.py
- chunker.py
- metadata.py

This provides clear separation of responsibilities and allows each stage to be tested or improved independently.

4.4 Local Embeddings

For semantic retrieval we used:

BAAI/bge-small-en-v1.5

Using a local embedding model reduced external API dependency and allowed repeated indexing and retrieval experiments during development.

What Works, Failed Approaches & Limitations

5. What Currently Works

The implemented prototype supports the main workflow required for Sub-track 1B.

Multi-format ingestion: PDF, DOCX, Markdown, and plain-text documents can be processed through dedicated loaders.

Preprocessing: Loaded content is cleaned, divided into retrievable chunks, and associated with metadata.

Vector retrieval: User questions are embedded and matched with semantically related document chunks.

Graph retrieval: Relevant entities can be identified in the graph and their neighboring relationships retrieved.

Graph expansion: The system can continue from initially retrieved entities to related entities instead of stopping at the first match.

Hybrid retrieval: Graph results and vector-search results are combined to provide broader evidence for answer generation.

A retrieval result can therefore include relationships such as:

Ederon Fellgard

- The Iron-Ring Cartel
- Greyfell Citadel
- Sapper
- related registry/service records

6. Approaches That Were Insufficient

Vector Retrieval Alone

Semantic vector retrieval successfully finds passages related to a question, but we found that it does not reliably reconstruct relationships distributed across several documents.

Increasing the number of retrieved chunks can provide additional evidence, but it also introduces irrelevant context without guaranteeing that the required relationship chain is found.

Therefore, vector retrieval was retained as part of the system but supplemented with graph retrieval.

Independent Chunk Retrieval

Treating all chunks as unrelated units loses relationships between entities appearing in different documents. This limitation motivated our graph-based representation and neighbor-expansion approach.

7. Current Limitations

- Graph retrieval depends on the quality of extracted entities and relationships.
- Different aliases or variations of an entity name can reduce graph connectivity.
- Hybrid retrieval improves evidence coverage but cannot guarantee that every required fact is retrieved.
- Chunk size creates a trade-off between retrieval precision and preserving surrounding context.
- The current solution primarily targets textual and relational evidence; complete visual understanding of diagrams and scanned images is outside our selected Sub-track 1B scope.
- Conflicting archive sources are retrieved as evidence, but the current prototype does not implement a dedicated source-reliability scoring mechanism.

AI Usage Disclosure & Engineering

8. AI Usage Disclosure

AI tools were used during development as permitted by the competition's AI Usage Policy. The challenge explicitly allows AI coding assistants and chat models while requiring teams to disclose how they were used and retain human responsibility for technical decisions. We used AI assistance for:

- discussing system architecture;
- comparing retrieval approaches;
- generating and refining code;
- debugging implementation issues;
- reviewing modular project structure;
- improving documentation;
- explaining Git workflows;
- analyzing technical alternatives;
- preparing the final report.

AI suggestions were **not accepted blindly**.

The team decided:

- which sub-track to solve;
- how the solution should be structured;
- to combine graph and vector retrieval;
- how document ingestion should be modularized;
- which generated code should be retained or modified;
- how features should be integrated;
- whether retrieved results were correct;
- which limitations should be reported.

Development was iterative rather than a one-prompt generation process. Generated code and technical suggestions were reviewed and tested before being integrated into the final system.

The required AI conversation logs are included with the repository submission.

9. Engineering Practices

Development followed a feature-based Git workflow. Major functionality was developed separately before integration, including branches such as:

- feature/ingestion-pipeline
- feature/chunking-pipeline

Changes were committed using meaningful messages and integrated through the development branch. The project was divided into independent components instead of implementing all functionality in a single module.

This improves maintainability, debugging, testing, and reproducibility.

This approach aligns with the evaluation criteria, which specifically consider meaningful Git history, repository organization, modular design, code quality, documentation, and reproducibility.

Team Members & Contributions

Member 1 Denipitiyage C.N

Role: Problem Analysis & Requirements

- Analyzed the competition challenge and the three available sub-tracks.
- Identified the primary problem the team should address.
- Contributed to selecting **Sub-track 1B**.
- Defined the expected solution scope and requirements.
- Communicated the problems and requirements to the implementation team.

Member 2 – Vimukthi L.H.T.P

Role: Software Developer / Implementation

- Implemented major components of the proposed solution.
- Worked on document ingestion and preprocessing.
- Contributed to cleaning, chunking, and metadata processing.
- Implemented/integrated semantic vector retrieval.
- Participated in system integration and debugging.
- Contributed to producing the final working prototype.

Member 3 — Uthpala P.B.H

Role: Software Developer / Implementation

- Implemented major components of the proposed solution.
- Worked on graph-based retrieval and relationship handling.
- Implemented/integrated graph-neighbor expansion.
- Contributed to hybrid graph and vector retrieval.
- Participated in system integration and debugging.
- Contributed to producing the final working prototype.

Member 4 — Jayaweera M.K.D

Role: Testing & Quality Assurance

- Performed testing after completion of the integrated implementation.
- Tested the system using representative questions and inputs.
- Verified end-to-end behavior of the final prototype.
- Identified retrieval and integration problems.
- Reported issues found during final testing.
- Verified the final system before submission.

10. Conclusion

Our solution addresses Sub-track 1B through a hybrid retrieval architecture that combines semantic vector search with graph-based relationship expansion. Vector retrieval identifies relevant textual evidence, while graph retrieval enables the system to follow relationships across documents. Their combination provides a practical method for answering questions whose evidence is distributed throughout the Ashen Era Archive. The current prototype has limitations, but it establishes a working foundation for multi-document reasoning and can be extended in the future with improved entity resolution, graph traversal, reranking, source-reliability analysis, and multimodal retrieval.